ATML Assignment 0: Foundations in Deep Learning

Sayim Mushtaq
LUMS
28100500@lums.edu.pk
https://github.com/sayim665/atml-pa0

Abstract

Accuracy numbers alone don't really tell you much about what's going on inside a model, so this report is less about how well each architecture performs and more about why it behaves the way it does. ResNet-152 gets used as a case study for transfer learning: we check how much of its performance on a new dataset comes purely from frozen, pretrained features versus actual fine-tuning, and we intentionally break its skip connections to see what they're really doing beyond just helping gradients flow. From there we turn to Vision Transformers, pulling out the raw attention weights behind a classification decision, testing how much of an image can go missing before predictions start falling apart, and checking whether the CLS token or a plain average over patch tokens ends up carrying more useful information. CLIP comes next, where the interesting part isn't just how well zero shot classification works but why prompt wording changes the result, and why its image and text embeddings end up sitting in noticeably different regions of the same space, along with whether a simple rotation can actually pull them back together without hurting accuracy. The last piece is a variational autoencoder built from scratch for MNIST, where we dig into what its latent space looks like, how faithfully it reconstructs digits, what it generates from pure noise, and how our design choices stack up against Doersch's reference implementation. Across all four, the goal each time is the same: not just reporting a number, but explaining the mechanism that produced it.

1 Introduction

It's easy to treat a deep learning model like a black box. You feed it data, check the accuracy, and move on. That's fine if all you care about is a number on a leaderboard, but it doesn't actually explain anything. It doesn't tell you why a model fails on certain inputs, why one design choice works better than another, or what the model has actually learned to represent internally. This report tries to do the opposite for four architectures, each one representing a genuinely different idea in deep learning: convolutional networks with residual connections (ResNet-152), self-attention based vision models (Vision Transformers), contrastive multimodal models (CLIP), and generative latent-variable models (variational autoencoders).

Instead of just using each of these as a tool, we treat each one as something worth studying in its own right. For ResNet-152, we go inside the network itself: disabling parts of it, pulling out intermediate features, and comparing different training setups, all to understand what the architecture is actually contributing rather than just how well it performs. For ViT, we take advantage of the fact that attention is something you can actually look at directly, unlike the internal representations of a CNN, so we use it to see exactly what the model is paying attention to when it makes a decision, and then stress-test that decision by removing information from the input. For CLIP, we look past the headline zero-shot accuracy number and dig into the geometry of its embedding space, which turns out to behave in a less intuitive way than the accuracy alone would suggest. For the VAE, we build the model ourselves

starting from its underlying probabilistic formulation, and use its latent space as a way to understand both what it captures well and where it starts to run out of room.

Each of the four sections that follow is structured roughly the same way: we describe the setup, report what we found, and then spend most of the space trying to explain why we think that result happened, connecting it back to earlier results in the same section wherever we can, or to ideas from the assignment's own reading list. At the end, we pull together a short summary of what these four otherwise very different experiments actually have in common.

2 Task 1: Inner Workings of ResNet-152

2.1 Methodology

All four experiments in this section use a ResNet-152 pretrained on ImageNet-1k, evaluated on CIFAR-10. Since ResNet-152 expects 224×224 inputs, we resize CIFAR-10's 32×32 images up to 224×224 and normalize them with ImageNet's channel statistics, so the pretrained weights see roughly the input distribution they were trained on. We use the Adam optimizer with cross-entropy loss throughout.

2.2 Baseline Setup

We replaced ResNet-152's final 1000-way ImageNet layer with a fresh linear layer mapping its 2048-dimensional feature vector to CIFAR-10's 10 classes, and froze the entire backbone so only this new head could train. The idea was to check whether the backbone's existing features, left completely untouched, were already good enough to separate CIFAR-10's classes with nothing more than a linear layer on top.

Training this head for 5 epochs on the full 50,000-image training set gave the results in Table 1. Only 20,490 of the model's 58,164,298 parameters, about 0.035%, were ever updated.

Table 1: head-only training on CIFAR-10, frozen ResNet-152 backbone.

Epoch	Train Loss	Train Acc.	Val. Acc.
1	0.4151	85.9%	85.3%
2	0.4067	86.2%	85.3%
3	0.4004	86.4%	85.2%
4	0.3933	86.6%	85.4%
5	0.3902	86.7%	84.9%

ResNet-152 is a huge network with over 58 million parameters and networks this size are really only trained from scratch when there's a dataset like ImageNet backing them, with over a million images to learn from. That's what lets the model figure out low-level features such as edges, textures, and shapes on its own. CIFAR-10 only has 50,000 training images, which just isn't enough data (or worth the compute) to train something this large from zero. In this baseline experiment, we froze the entire backbone and trained only a new linear head, 20,490/58,164,298 parameters. Despite this, the model reached 84.93 percent accuracy on CIFAR-10 after 5 epochs, a dataset with entirely different classes from ImageNet. This explains that the hierarchical features ResNet-152 learned during ImageNet pretraining (edges, textures, shapes and object parts) are general enough to be directly reusable for a new classification task.

2.3 Residual Connections in Practice

To see what skip connections actually contribute, we picked two consecutive bottleneck blocks in `layer3` (`layer3.0` and `layer3.1`, chosen from this stage specifically because, with 36 blocks, it is the deepest and the one most associated with vanishing gradients) and rewrote their forward pass to drop the identity addition, leaving everything else about the block untouched. We then trained a new head on top of this modified, still-frozen backbone, using the exact same setup as the baseline.

The result was dramatic: validation accuracy collapsed to somewhere around 18–20%, barely above the 10% floor you'd expect from random guessing across 10 classes, compared to the intact baseline's

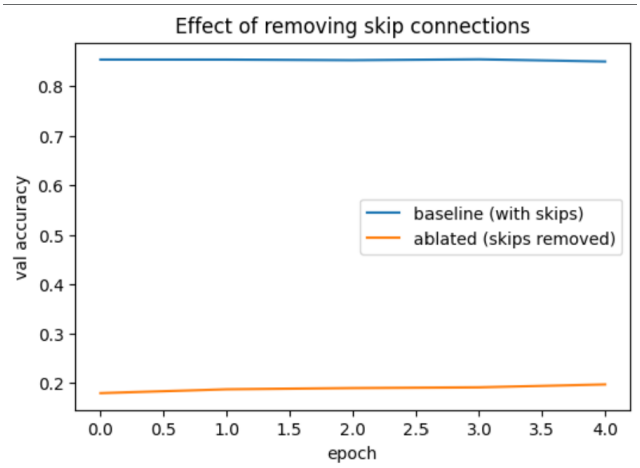

Figure 1: validation accuracy across training for the intact baseline versus the model with skip connections removed from two blocks in `layer3`.

stable ∼85% (Figure 1). The textbook explanation for why skip connections matter is about gradients: the identity path gives gradients a direct route backward through very deep networks, which is exactly the problem ResNet was designed around. But that explanation doesn't quite fit what happened here, because the backbone was frozen the whole time, so no gradients were ever flowing through those blocks in the first place, only through the new head. What's actually going on is more of a forward-pass problem. The convolutional weights inside those two blocks were trained under the assumption that their output would get added back to their input. Take that addition away, and the features coming out of the block are something the rest of the network was never trained to make sense of. That corrupted signal then just flows straight through every remaining frozen layer with no chance to recover. So skip connections aren't only helping with training dynamics; they're baked into what the weights actually represent, and removing them breaks the network's computation even without touching gradients at all.

2.4 Feature Hierarchies and Representations

Using forward hooks, we pulled out activations from three points in the network: `layer1` (early), `layer3` (middle), and `layer4` (late), for roughly 1,000 CIFAR-10 test images, averaged each layer's output spatially into one vector per image, and ran t-SNE on each set separately to see how the classes were arranged.

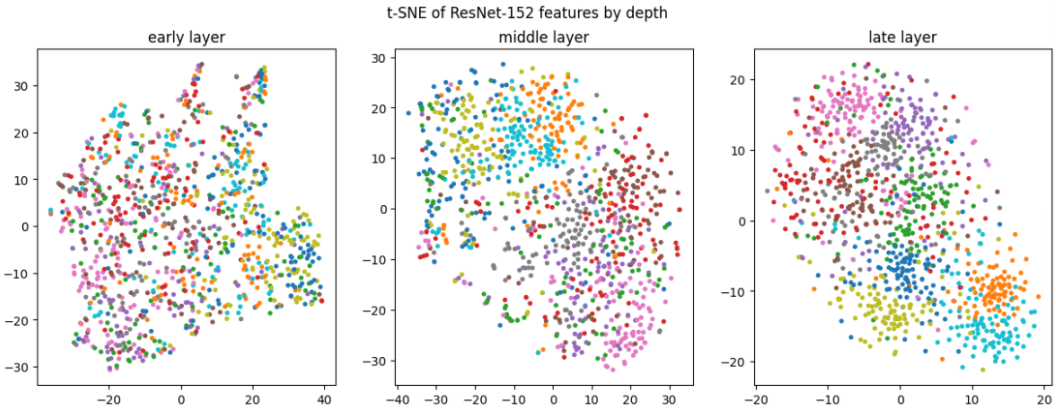

Figure 2: t-SNE of ResNet-152 features at early, middle, and late depth, colored by CIFAR-10 class.

Looking at the three plots, (Figure 2) theres a clear progression as you go deeper into the network. In early layer, the points are basically a scattered mess with colors all mixed together with no clear

grouping, which makes sense since this stage of the network is just picking up on generic stuff like edges ,colors and simple textures that show up in pretty much every image, regardless of what class it belongs to. By the middle layer we can see some loose clumping happening, then in the late layer, the seperation is much more obvious, you can actually pick out distinct groups now. This tracks with how CNNs generally work, the early layers extract low level features, while the deeper layers combine those into higher-level, more abstract features that actually correspond to object identity.

2.5 Transfer Learning and Generalization

We compared two separate design choices, both using a fixed 5,000-image subset of CIFAR-10 and 3 training epochs, to keep the comparisons tractable: pretrained versus random initialization with everything else held fixed, and fine-tuning only the last residual stage (`layer4` plus the head) versus fine-tuning the whole backbone.

Table 2: comparison of fine-tuning configurations on CIFAR-10.

Configuration	Trainable Params	Data	Time/Epoch	Val. Acc.
Head-only (Sec. 1.2 baseline)	20,490 (0.035%)	Full (50k)	$\sim$8.5 min	84.9%
Full fine-tune, pretrained init	58.16M (100%)	Subset (5k)	$\sim$2.3 min	94.1%
Full fine-tune, random init	58.16M (100%)	Subset (5k)	$\sim$2.4 min	26.7%
Last-block only (`layer4`+head)	$\sim$8.7M ($\sim$15%)	Subset (5k)	$\sim$55 sec	85.5%

The pretrained-versus-random comparison makes the value of pretraining obvious immediately: the pretrained model hit 91.0% validation accuracy after a single epoch, while the randomly initialized version reached only 19.0% after that same epoch and just 26.7% after all three, still climbing slowly. With everything else identical, the only difference is the starting weights. The pretrained model only has to *adapt* an already strong visual representation, while the random one has to learn basic visual primitives from almost nothing, which a network this size simply can't do in three epochs on five thousand images.

Looking at all four setups together, the last-block configuration really stands out as the sweet spot. It got basically the same accuracy as the head-only baseline, around 85%, but did it in a fraction of the time (under a minute per epoch compared to roughly 8.5 minutes for the head-only run), and on top of that, it used a much smaller subset of data (5,000 images instead of the full 50,000). Full fine-tuning of the pretrained model did reach the highest accuracy overall at 94.1%, but that came at a real cost: all 58 million parameters had to be updated, and each epoch took noticeably longer than the last-block setup, even with far less training data involved. This lines up well with what we saw earlier in the feature hierarchy plot (Section 1.4): the early and middle layers weren't doing much in the way of class-specific work; it was the fourth layer where features started becoming meaningfully class-discriminative. So it makes sense that unfreezing just `layer4` recovers almost all the accuracy you'd get from fine-tuning the entire network. Following that logic, the earlier layers (`layer1` through `layer3`) end up being the most transferable across datasets, while `layer4` is the least transferable, since it's the part of the network that actually needs to adapt to the new task.

3 Task 2: Understanding Vision Transformers

3.1 Methodology

We used `google/vit-base-patch16-224`, pretrained on ImageNet-21k and fine-tuned on ImageNet-1k, through HuggingFace's `transformers` library. A 224×224 image gets split into 16×16 patches, giving $14 \times 14 = 196$ patch tokens, plus one learnable CLS token appended at the front, for 197 tokens total. Attention weights were extracted using the model's `eager` attention implementation, since the default fused implementation does not return them.

3.2 Classification and Attention Visualization

We classified two images, a golden retriever and a black-and-white cat, and for each one pulled the final layer's attention tensor (shape $(1, 12, 197, 197)$), averaged it over the 12 heads, and took the

CLS token's attention row to the 196 patch tokens. That vector was reshaped into a 14×14 grid and upsampled back to 224×224 to overlay on the original image.

The dog was classified as "golden retriever" with 86.5% confidence, essentially a perfect prediction, matching what's clearly in the photo. The cat came back as "Egyptian cat" with only 37.9% confidence, which is a more questionable call: the actual image shows a black-and-white bicolor cat, while "Egyptian cat" in ImageNet's taxonomy usually refers to a fairly different, brownish/spotted coloring, and the low confidence reflects that the model itself wasn't sure.

Figure 3: CLS-token attention overlays, final layer, averaged over heads, for the dog and cat images.

The attention maps back this up nicely (Figure 3). For the dog image, the attention was mostly concentrated right around the mouth and the stick it was holding, which is a pretty specific detail rather than the whole animal, but it still landed on the dog itself, so the model was clearly picking up on something relevant. The cat image told a different story: most of the strong attention ended up on the background rather than the cat's face or body, and only a small hot spot in the bottom right actually landed near the cat. This lines up with an earlier result too, since the cat prediction was both less confident (37.9%) and arguably less accurate than the dog prediction, so it makes sense that the attention map for the cat looks messier and less focused on the actual subject.

Compared to something like CAM or Grad-CAM, which have to work backwards from gradients flowing into the last convolutional layer to figure out which pixels mattered, ViT's attention is just sitting there natively as part of the forward pass. That's a real advantage for interpretability: it's cheaper to compute, and it's tied directly to the actual mechanism the model uses to make its decision, rather than being an approximation reconstructed after the fact.

Looking at the 12 heads separately rather than averaged (Figure 4) shows they're clearly not all doing the same thing. Most look scattered and noisy, but a couple (heads 1 and 8 in our case) show noticeably more localized or broader coherent patterns than the rest. That's direct evidence the heads are specialized rather than duplicating each other.

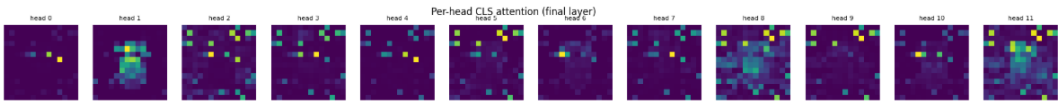

Figure 4: final-layer CLS-token attention for each of the 12 heads separately, dog image.

3.3 Patch-Masking Robustness

Since STL-10's 10 classes don't map cleanly onto ImageNet's 1000 categories, we measured robustness as agreement with the model's own unmasked prediction, rather than accuracy against ground truth. We tested 100 STL-10 images at mask fractions of $0, 0.1, 0.25, 0.4, 0.6$, and 0.8, comparing random patch masking against structured masking (patches nearest the center masked first).

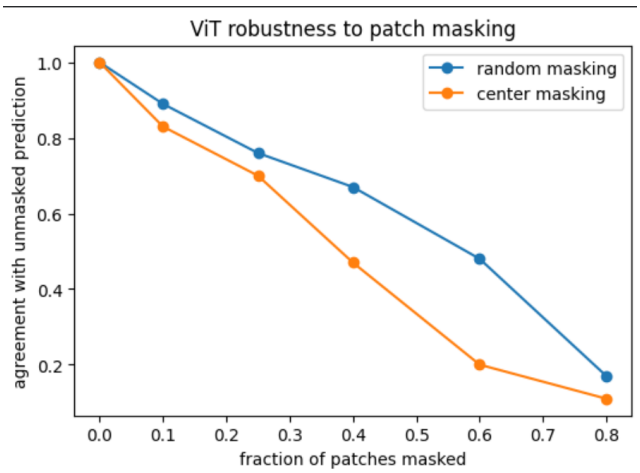

Figure 5: agreement with the unmasked prediction as a function of mask fraction, random vs. center masking.

The ViT handles random masking pretty well,(Figure 5). at least up to a point: even with 40% of the patches blacked out, its prediction still agreed with the unmasked one about 67% of the time. Center masking is a different story though, and it falls apart much faster: at that same 40% mark, agreement drops to under 50%, and by 60% masked it's already down around 20%, well below where random masking is at that point.

The reason for this gap comes down to where the missing information actually is. Random masking spreads the missing patches out across the whole image, so even if you lose 40% of them, there's usually still enough scattered elsewhere for the model to piece together what it's looking at. This fits with what we saw in the attention maps earlier, where the CLS token wasn't just locking onto one tiny spot but was drawing information from multiple regions. Center masking, on the other hand, specifically wipes out the middle of the image, and since most photos (including the STL-10 ones here) have their main subject roughly centered, this is basically the worst-case scenario: you're removing exactly the part of the image most likely to contain the actual object. So it's not really that the model is "more fragile" under center masking; it's that center masking is specifically targeting the most informative region, while random masking just thins things out more evenly.

3.4 CLS Token vs. Mean-Pooled Patch Tokens

We extracted both the CLS token's final-layer output and the mean of all 196 patch tokens for 500 STL-10 training images, then trained two separate logistic regression probes, one per pooling method, on a 70/30 split using STL-10's real labels.

Both the CLS-token probe and the mean-pooled probe hit 100% accuracy, so there's no real winner here; they performed identically. That's probably less about the pooling method not mattering in general, and more about this particular task being too easy for it to show up as a difference. STL-10's

classes (dog, cat, airplane, etc.) overlap heavily with what this ViT was already trained to recognize on ImageNet, so by the time you're extracting features from a model this strong, either pooling strategy already carries more than enough signal for a simple linear classifier to separate 10 classes perfectly. On top of that, the test set here was small (only around 150 images after the split), which makes it even easier for both to land at a perfect score just by chance.

As for how pooling might interact with the pretraining objective, the CLS token is the one that actually gets used during classification-style pretraining, so it's specifically shaped to summarize whatever's relevant for predicting the class. Mean-pooling isn't optimized for that directly; it's just an average over token representations that were mainly learned to encode more local, patch-level information. So in principle you'd expect CLS to have an edge on classification tasks, especially ones that are harder or more out-of-distribution than STL-10. But for self-supervised or contrastive pretraining setups (something like DINO, for example), that relationship can flip, since those objectives don't always give the CLS token the same special status; sometimes the patch-level or mean-pooled representations end up carrying just as much, or more, useful information. Given our result was a tie, it's hard to say much more than that the two are interchangeable here, but that's likely a reflection of how easy this particular task was rather than a general statement about pooling strategies.

4 Task 3: CLIP

4.1 Methodology

We used OpenAI's official CLIP implementation with the ViT-B/32 backbone, evaluated on STL-10's 8,000-image test set across 10 classes, using CLIP's own preprocessing throughout. Since STL-10 has class labels rather than captions, we treated each image's true label, wrapped in a prompt template, as its paired "caption" for Sections 3.2 and 3.3.

4.2 Zero-Shot Classification

We compared three prompting strategies, a plain label, the template "a photo of a {class}", and a longer descriptive template, on the complete STL-10 test set.

Table 3: CLIP zero-shot accuracy on the full STL-10 test set by prompting strategy.

Prompting strategy	Accuracy
Plain label	96.26%
"a photo of a {class}"	**97.36%**
Descriptive template	96.71%

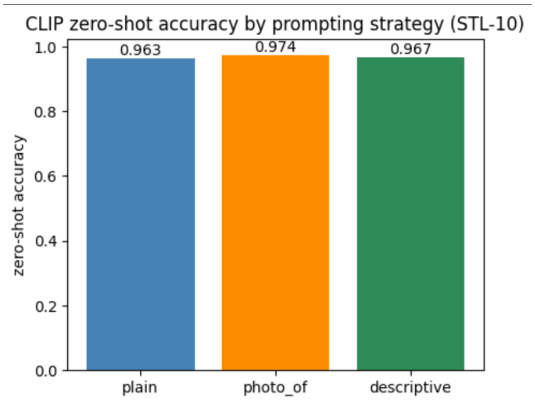

Figure 6: zero-shot accuracy on STL-10 by prompting strategy.

All three strategies land above 96%, which on its own says a lot about how strong CLIP's zero-shot ability is given it was never trained on STL-10 (Table 3, Figure 6). Still, "a photo of a {class}"

comes out ahead at 97.36%, beating both the bare label and the more elaborate description. CLIP was trained on real internet image-caption pairs, which tend to be short, natural phrases, not single bare words, and not unusually long descriptions either. A plain label is too sparse relative to what CLIP actually saw during training, while the descriptive prompt overshoots in the other direction; the templated version sits closest to CLIP's real training distribution, which is likely why it wins.

4.3 The Modality Gap

We pulled image and (label-derived) text embeddings for 100 randomly sampled STL-10 test images, projected both sets together into two dimensions with t-SNE, and computed two numbers: the distance between the image and text centroids, and the mean cosine similarity of matched image-text pairs.

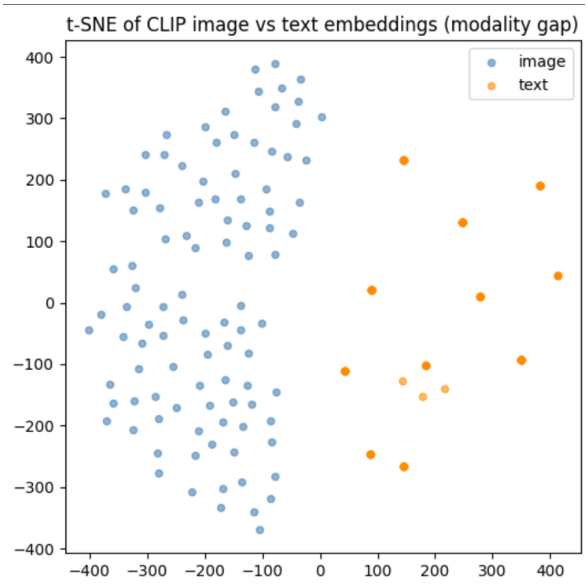

Figure 7: t-SNE of CLIP image and text embeddings, before alignment.

Looking at the t-SNE plot 7, the separation is pretty stark: the blue (image) points sit almost entirely on the left side, while the orange (text) points cluster off to the right, with basically no mixing between the two groups. The numbers back this up: the centroid distance between the average image embedding and average text embedding is 1.05, which is a big gap considering these are unit-normalized vectors (max possible distance is 2, and two totally random vectors would typically land around 1.4 apart). Even more telling is the cosine similarity for genuinely matched pairs, an actual cat photo compared against the text "a photo of a cat," which only averages 0.26. That's a long way from 1.0, which is what you'd expect if the model treated a matching image and caption as basically pointing in the same direction.

As for normalization, both sets of embeddings here were already L2-normalized (unit vectors), which is standard for CLIP since its whole training setup is built around cosine similarity. Normalizing removes any difference in vector length between the two modalities, but it doesn't do anything about the direction they're pointing in, and that's exactly where the gap shows up. So the separation isn't some leftover artifact of one modality having bigger numbers than the other; it's a genuine directional offset baked into the embedding space itself, and normalization doesn't erase it.

Given all that, it might seem surprising that CLIP still classifies almost everything correctly, but that's because zero-shot classification doesn't need the matched pair's similarity to be close to 1. It just needs the correct class's similarity to be higher than every other class's similarity for that same image. Even at a modest 0.26 average, that's still consistently higher than the similarity to the wrong classes' text embeddings, which is all the model actually needs to pick the right one. So the modality gap is more like a fixed offset sitting between the two spaces: it doesn't line up images and captions on top of each other, but it doesn't scramble the relative ordering of similarities either, and that relative ordering is really all zero-shot classification is built on.

4.4 Bridging the Modality Gap

We used `scipy.linalg.orthogonal_procrustes` to find the single rotation matrix R minimizing $\|XR - Y\|_F$ over the 100 matched pairs from Section 3.2, then re-visualized the aligned embeddings and recomputed zero-shot accuracy on the full 8,000-image test set, applying R to the image embeddings and comparing against the original text embeddings.

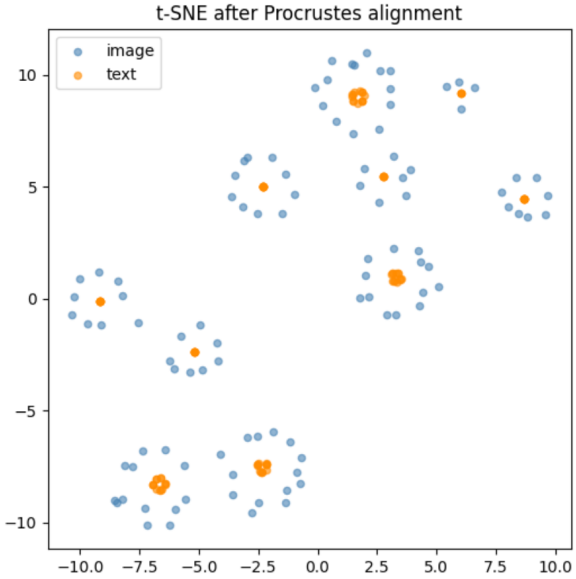

Figure 8: t-SNE of CLIP embeddings after Procrustes alignment.

The alignment clearly did what it was supposed to on the geometry side. The Frobenius norm between the image and text embeddings dropped from 12.15 down to 5.10, more than halved, and the t-SNE plot after alignment (Figure 8) backed this up visually too: instead of two separate blobs, you get ten tight little clusters, each with a text embedding sitting right in the middle surrounded by its matching images. So geometrically, the modality gap really did shrink a lot.

But that improvement didn't carry over to classification accuracy. If anything, it went slightly the other way, dropping from 97.36% before alignment to 96.86% after. The most likely explanation is overfitting: the rotation matrix R was only fit using 100 image-text pairs (10 per class), so while it does a great job of pulling those specific 100 points together, it's tuned to the particular quirks of that small sample rather than the full 8,000-image test set. When applied more broadly, a rotation learned from so little data doesn't generalize quite as well, so some images end up slightly worse-aligned than they would have been without any rotation at all. It's also worth pointing out that the drop was small, and the starting point was already really strong: 97.36% zero-shot accuracy left very little room for a geometric fix to meaningfully improve things, especially one that's at risk of overfitting to begin with.

So the bigger takeaway here isn't really that alignment failed; it's that visually or geometrically closing the modality gap and improving actual task performance aren't necessarily the same thing. A rotation can make embeddings look much better aligned in a plot without translating into better classification, particularly when it's been learned from a sample too small to represent the full data distribution well.

5 Task 4: Variational Autoencoders

5.1 Methodology

We built a VAE for MNIST with a simple MLP encoder and decoder. The encoder takes a flattened 784-dimensional image through a 400-unit hidden layer (ReLU) to a mean $\mu_\phi(x)$ and log-variance $\log \sigma^2_\phi(x)$ for the approximate posterior. Latent samples come from the reparameterization trick,

$z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, which moves the randomness into ϵ so gradients can still flow through μ and σ. The decoder mirrors the encoder, mapping z back through a 400-unit hidden layer to 784 pixel logits, treating each pixel as an independent Bernoulli variable. The loss is the negative ELBO: binary cross-entropy reconstruction loss plus the closed-form KL divergence against $\mathcal{N}(0, I)$, trained with Adam (learning rate 10^{-3}) for 15 epochs. We used latent dimension 2 first, as required for direct visualization, then trained a second model with latent dimension 10 for comparison.

5.2 Training

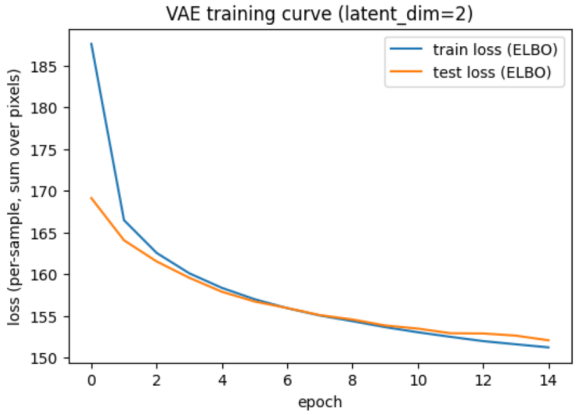

Figure 9: VAE training and test loss over 15 epochs, latent_dim=2.

Training was smooth (Figure 9): loss dropped from 187.62 to 151.22 on the training set, with test loss tracking closely the whole way (152.06 final), and no sign of overfitting. The reconstruction term fell substantially (181.81 to 145.28) while the KL term actually rose slightly (5.81 to 5.94), which is a healthy pattern: it means the model was actively using the latent space to encode useful information rather than collapsing everything toward the prior regardless of input.

5.3 Latent Space Structure

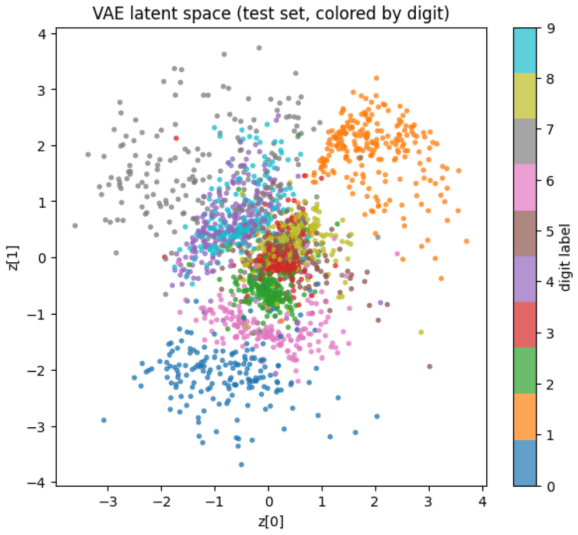

Figure 10: VAE latent space (test set, colored by digit), latent_dim=2.

Some digits form clean, distinct clusters (0, 1, and 7 in particular), while the rest (2, 3, 4, 5, 6, 8, 9) crowd together in a busy central region with a lot of overlap (Figure 10). That makes sense given the tight 2-dimensional bottleneck: there's only so much room to separate 10 classes plus all their natural

handwriting variation, so visually simple, distinctive digits get their own space while visually similar ones compete for the same territory. The smooth blending between neighboring clusters, rather than sharp boundaries, reflects the KL term pulling everything toward one shared prior: points sitting between two clusters likely decode into something visually "in-between" rather than snapping cleanly to one class or the other.

5.4 Reconstruction Quality

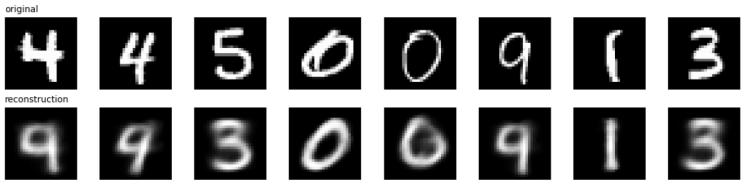

Figure 11: original (top) vs. reconstructed (bottom) digits, latent_dim=2.

Digits from the well-separated latent regions (0, 1, 9, 3) reconstruct accurately, but digits from the crowded middle get reconstructed as their nearest neighbor instead: a "4" comes back looking like a "9," a "5" comes back looking like a "3" (Figure 11). This isn't a coincidence: it's the exact same digit pairs that were overlapping in the latent space plot, which is direct evidence that the 2-dimensional bottleneck is a genuine capacity limit and not just a cosmetic artifact of the visualization. Even the correct reconstructions come out somewhat blurry, which is typical of VAEs, since the KL term pushes the model toward smooth, averaged posterior distributions rather than sharp, confident point estimates.

5.5 Generation from the Prior

Figure 12: samples generated by decoding random draws from $\mathcal{N}(0, I)$, latent_dim=2.

Sampling random points from the prior and decoding them (Figure 12) gave a set dominated by "9"-looking digits, roughly 60% of the batch, with only a few samples covering other classes. This traces directly back to the latent space structure: since $\mathcal{N}(0, I)$ concentrates most of its probability mass near the origin, and the origin happens to sit right in that crowded, 9-dominated central region, random sampling disproportionately lands there. Digit 1, whose cluster sits further out, is correspondingly rare in generated samples even though it reconstructs perfectly when encoded directly. This is a useful distinction to flag: reconstruction quality and generative diversity aren't the same thing, and a model can be good at one while falling short on the other if the posterior over the training data doesn't evenly cover the latent space.

5.6 Effect of Latent Dimensionality

We trained a second VAE with latent dimension 10, everything else held fixed. Final test loss dropped from 152.06 to 108.52, a substantial improvement, and reconstructions that had previously failed (4 as 9, 5 as 3) became fully accurate across the board. Generated samples also became far more diverse, spreading across most digit classes instead of collapsing onto one shape.

5.7 Comparison with Doersch's VAE Implementation

Architecture. Like Doersch's reference implementation, we used a simple MLP for both the encoder and decoder rather than a convolutional architecture, a reasonable choice for MNIST given how small and simple the images are. Our version uses a single 400-unit hidden layer with ReLU activations on both sides, which is roughly in the same range Doersch describes using in his tutorial.

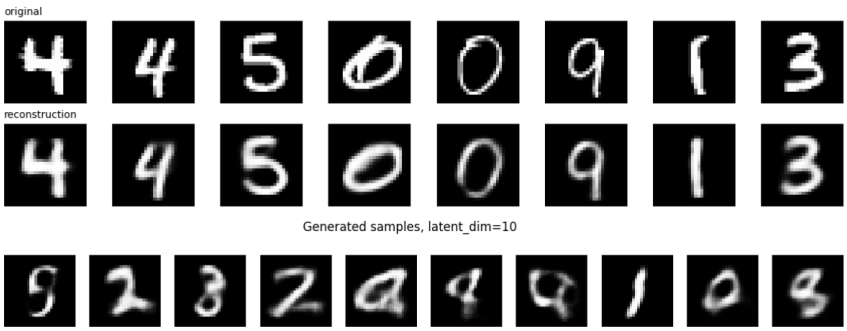

Figure 13: reconstructions and generated samples with latent_dim=10, alongside the final test loss comparison against latent_dim=2.

This is a pretty standard, uncomplicated setup, and it's clearly enough to get a working VAE, though obviously a deeper network or a convolution-based encoder and decoder could probably squeeze out a bit more reconstruction quality.

Output distribution and loss. This matches almost exactly. Doersch treats pixel intensities as probabilities and uses sigmoid cross-entropy loss, which amounts to a Bernoulli likelihood per pixel, and that's precisely what we implemented too, using `binary_cross_entropy_with_logits` for the reconstruction term. Since MNIST pixels are naturally in $[0, 1]$ after normalization, this is a natural fit for both implementations, and it's a big part of why the reconstructions come out in that soft, grayscale-blurred style rather than looking noisy or discrete.

Latent dimensionality. We started with latent_dim=2 specifically because the assignment asks for it, mainly for the sake of being able to plot the latent space directly without needing any dimensionality reduction. We then trained a second model with latent_dim=10 to see what changed, and the difference was substantial: final test loss dropped from 152.06 down to 108.52, reconstructions that were previously getting confused (4 read as 9, 5 read as 3) became fully accurate, and the generated samples went from being dominated by "9"-looking shapes to covering a much wider spread of digit classes. This lines up well with what Doersch says about VAEs being relatively insensitive to latent dimensionality over a broad range, except that going too small (like our 2D case) clearly does hurt things here, since 2 dimensions just isn't enough room to cleanly separate 10 digit classes plus their natural handwriting variation.

Results comparison. Doersch's tutorial notes that his generated digits mostly look realistic, with a handful of "in-between," ambiguous ones showing up, and we saw exactly the same pattern in our own generations, especially with the smaller latent_dim=2 model, and even a little in the latent_dim=10 batch (a couple of the generated samples came out looking like blends between two digits, kind of like something between an "a" and a "6"). This kind of failure case, blurry, in-between-looking outputs, seems to be a pretty inherent property of VAEs in general rather than something specific to either implementation, and it tracks with what we saw in our own reconstructions: whenever the latent space is crowded or two classes are close together, the decoder tends to produce something that splits the difference rather than confidently committing to one class or the other.

6 Conclusion

Across all four tasks, the same pattern kept showing up: understanding a model's behavior comes down to connecting its internal representations to what it actually does downstream. In Task 1, how ResNet-152's features changed with depth explained both why frozen-backbone transfer learning worked so well and which layers were worth fine-tuning. In Task 2, ViT's distributed attention explained why it degraded gracefully under random masking but collapsed fast under center masking, and attention quality tracked directly with prediction confidence. In Task 3, CLIP's persistent modality gap didn't stop zero-shot classification from working, since classification only needs relative similarity ordering to hold, and closing that gap geometrically via Procrustes alignment didn't translate into

better accuracy. In Task 4, the VAE's latent space structure directly predicted both its reconstruction failures and the limits of what it could generate, with latent dimensionality turning out to be the key lever connecting the two.

References

[1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*.

[2] Dosovitskiy, A., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*.

[3] Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. *ICML*.

[4] Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *ICLR*.

[5] Doersch, C. (2016). Tutorial on Variational Autoencoders. *arXiv:1606.05908*.